



UNIVERSIDAD AUTÓNOMA METROPOLITANA
Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación



Hallazgos de la prueba de despliegue desde cero

Proyecto Tecnológico

Bitácora de instalación versión

Trimestre 2026-Primavera

[TU NOMBRE]

[MATRÍCULA]

al[MATRÍCULA]@azc.uam.mx

[Grado. Nombres Apellidos del Asesor]

[Profesor Titular/Asociado/Asistente]

[Departamento de Sistemas]

[correo@azc.uam.mx]

21 de agosto de 2026

Declaratoria

En caso de que el Comité de Estudios de la Licenciatura en Computación apruebe la realización de la presente propuesta, otorgamos nuestra autorización para su publicación en la página de la División de Ciencias Básicas e Ingeniería.

[TU NOMBRE]

[Grado. Nombres Apellidos del Asesor]

1. Contexto de la prueba

Este documento acompaña al *Manual de Instalación* (`main.pdf`) como bitácora de una prueba de despliegue *desde cero* realizada en una máquina limpia (equivalente a lo que hará el sinodal al reproducir el proyecto). Su propósito es doble: (1) dejar evidencia de que el sistema completo se monta siguiendo el manual y (2) documentar las discrepancias detectadas entre el manual publicado y el estado real del repositorio para poder corregirlas.

Elemento	Valor
Fecha de la prueba	2026-08-21
Repositorio	https://github.com/Aldebaran-GR/proyecto_terminal_cyad.git
Commit probado	eac369d99e77fdee6e1443cfc72d5ba62341a690
Sistema operativo	Windows 11 Home 10.0.26200
Shells utilizados	PowerShell 5.1 y Git Bash 2.47
Modalidad	Clon paralelo con <code>git clone -depth 1</code>
Base de datos	PostgreSQL 17, base <code>proyecto_terminal_cyad_test</code> (aislada)

2. Prerrequisitos verificados

Salidas literales de los comandos de verificación de versión:

```
$ python --version
Python 3.13.0

$ node --version
v22.16.0

$ npm --version
10.9.2

$ git --version
git version 2.47.0.windows.2

$ & 'C:\Program Files\PostgreSQL\17\bin\psql.exe' --version
psql (PostgreSQL) 17.0
```

Todos los prerrequisitos del manual están cubiertos por versiones iguales o superiores. Se encontró un detalle sobre `psql` en Windows que ameritó ampliar el manual (ver hallazgo H2).

3. Rutas utilizadas

Recurso	Ruta
Carpeta padre de la prueba	C:\Users\godin\Documents\10_2025_-Proyectos\prueba_despliegue\
Clon del repositorio	prueba_despliegue\proyecto_terminal_cyad\
Entorno virtual Python	...\backend\.venv\
Archivo .env	...\backend\.env
Ejecutable psql	C:\Program Files\PostgreSQL\17\bin\psql.exe
Servicio PostgreSQL	postgresql-x64-17 (estado <i>Running</i>)
Base de datos	proyecto_terminal_cyad_test
Backend (dev server)	http://localhost:8000
OpenAPI Swagger UI	http://localhost:8000/api/docs/
Frontend (Vite dev)	http://localhost:5173

4. Cronología por fase

Fase	Comando principal	Duración	Resultado
A	Verificación de prerequisites	–	OK (5/5)
B	git clone -depth 1	2.2s	OK (13 MB)
C	python -m venv .venv	<3s	OK
C	pip install -r requirements.txt	53.2s	OK, 28 paquetes
C	python scripts/create_db.py	<1s	OK
C	python manage.py migrate	4.5s	OK, 38 migraciones
C	python scripts/seed_demo.py	<4s	FALLA (H5), fix aplicado, 3.2s en 2.º intento
D	npm install	11.3s	OK, 194 paquetes (8 <i>vuln</i> — ver H6)
D	npm run build	5.9s	OK (263 KB gzip)
D	npm run dev	inmediato	OK (HTTP 200 en /)
E	python scripts/smoke_test.py	30.8s	OK (12/12 checks)

5. Transcripción de comandos

5.1. Fase B — Clonado

```
$ git clone --depth 1 https://github.com/Aldebaran-GR/proyecto_terminal_cyad.git
Cloning into 'proyecto_terminal_cyad'...
real 0m2.224s
```

```
$ du -sh proyecto_terminal_cyad
13M  proyecto_terminal_cyad

$ git -C proyecto_terminal_cyad rev-parse HEAD
eac369d99e77fdee6e1443cfc72d5ba62341a690
```

5.2. Fase C — Backend

Entorno virtual y dependencias.

```
$ cd backend
$ python -m venv .venv
$ .venv/Scripts/python.exe -m pip install --upgrade pip
Successfully installed pip-26.2.1
real  0m12.685s

$ .venv/Scripts/python.exe -m pip install -r requirements.txt
Successfully installed Django-5.1.6 Faker-40.37.0 PyYAML-6.0.3 asgiref-3.12.1
attrs-26.1.0 colorama-0.4.6 django-cors-headers-4.6.0 django-filter-24.3
djangorestframework-3.15.2 djangorestframework-simplejwt-5.3.1
drf-spectacular-0.28.0 factory-boy-3.3.1 inflection-0.5.1 iniconfig-2.3.0
jsonschema-4.26.0 jsonschema-specifications-2025.9.1 packaging-26.3
pluggy-1.6.0 psycpg2-binary-2.9.10 pyjwt-2.13.0 pytest-8.3.4
pytest-django-4.9.0 python-decouple-3.8 referencing-0.37.0
rpds-py-2026.6.3 sqlparse-0.6.0 tzdata-2026.3 uritemplate-4.2.0
real  0m53.184s
```

Configuración de .env. El .env se generó copiando el existente del proyecto original y cambiando únicamente DB_NAME a proyecto_terminal_cyad_test para no colisionar con datos previos. La contraseña de PostgreSQL nunca se imprimió en pantalla ni se registró en esta bitácora.

Base de datos y migraciones.

```
$ python scripts/create_db.py
Base de datos 'proyecto_terminal_cyad_test' creada exitosamente.
Conexión a PostgreSQL: OK

$ python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, auth, autoevaluacion, catalogos,
                        contenttypes, documentos, sessions
Running migrations:
  Applying catalogos.0001_initial... OK
  ... [38 migraciones aplicadas]
  Applying sessions.0001_initial... OK
real  0m4.499s
```

Seed de datos (primer intento — fracaso).

```
$ python scripts/seed_demo.py
=== Departamentos, Licenciaturas y Periodo activo ===
[CREADO] Depto ICD, PTR, MA
[CREADO] Lic DCG, ARQ, DI, DiPS
[CREADO] Posgrado PDB
[CREADO] Periodo 26-I
... [omitido: creación de áreas, UEAs, profesores, formulario]
[CREADO] Formulario Autoevaluación Docente 26-I
Preguntas, opciones y niveles creados.
Traceback (most recent call last):
  File ".../scripts/seed_demo.py", line 397, in <module>
    formulario.publicar()
  File ".../autoevaluacion/models.py", line 84, in publicar
    self._validar_estructura_para_publicar()
  File ".../autoevaluacion/models.py", line 69, in _validar_estructura...
    raise ValueError(...)
ValueError: La suma de pesos de las secciones debe ser 100 %
            (actual: 0.00 %). Ajusta los pesos antes de publicar.
```

Workaround aplicado. Se editó `scripts/seed_demo.py` para agregar `peso=Decimal("50")` a la creación de ambas secciones (*Planeación y Desempeño en aula*), sumando 100 %. Se ejecutó `python manage.py flush --noinput` para dejar la BD vacía y se relanzó el seed:

```
$ python scripts/seed_demo.py
... [creación exitosa de todos los objetos]
Formulario publicado (v1).

=== LISTO ===
Credenciales de prueba:
ADMIN      -> admin@cyad.uam.mx      / Admin1234!
PROFESOR   -> profesor@cyad.uam.mx   / Profesor1234!
PROFESOR   -> profesor2@cyad.uam.mx  / Profesor1234!
Periodo activo: 26-I      Formulario publicado: "Autoevaluación Docente 26-I"
real    0m3.169s
```

Servidor Django.

```
$ python manage.py runserver 8000          # ejecutado en segundo plano
$ curl -o /dev/null -w "HTTP %{http_code}\n" http://localhost:8000/api/docs/
HTTP 200
```

5.3. Fase D — Frontend

```
$ cd ../frontend
$ npm install
```

```
added 194 packages, and audited 195 packages in 10s
8 vulnerabilities (1 moderate, 7 high)      # ver hallazgo H6
real    0m11.305s

$ npm run build
vite v8.0.16 building client environment for production...
transforming... 262 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                0.47 kB | gzip:    0.30 kB
dist/assets/index-CFTd8wEv.css 35.44 kB | gzip:    7.10 kB
dist/assets/index-BLSt0mwu.js 893.28 kB | gzip: 263.33 kB
built in 1.78s
(!) Some chunks are larger than 500 kB after minification.

$ npm run dev                                     # ejecutado en segundo plano
$ curl -o /dev/null -w "HTTP %{http_code}\n" http://localhost:5173/
HTTP 200
```

6. Verificación en el navegador

Las siguientes capturas se obtuvieron con el clon fresco tras completar el seed de demostración corregido.

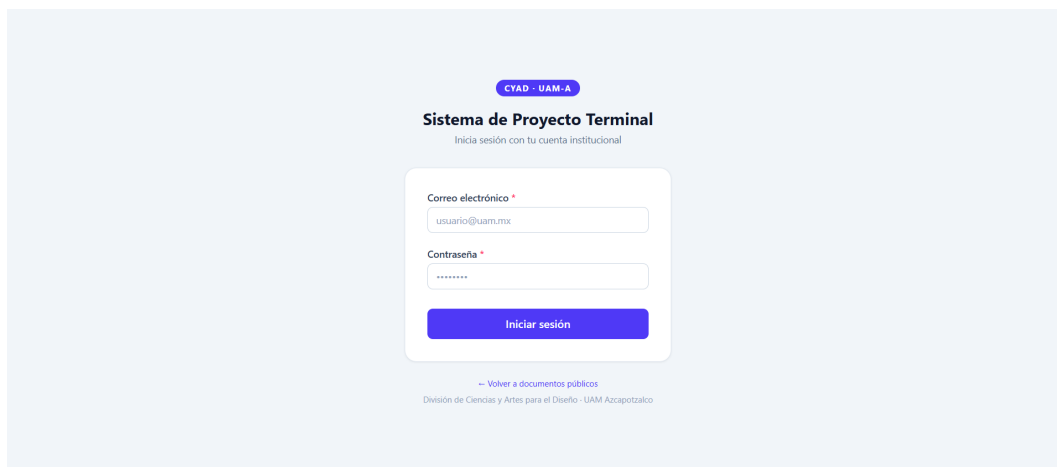


Figura 1. Vista pública inicial en `http://localhost:5173/` — enlaces a documentos públicos y al inicio de sesión.

Figura 2. Formulario de inicio de sesión con las credenciales de administrador.

Dashboard
Bienvenido, Administrador CyAD.

Profesores activos
2

Cartas temáticas
0
Publicadas

Requisitos
0
Publicados

Formularios publicados
1

Cartas Temáticas
26-I

CUMPLIMIENTO POR DEPARTAMENTO

DEPARTAMENTO	PROFESORES	CARTA
EDT Evaluación del Diseño en el Tiempo	0/0	0
ICD Investigación y Conocimiento para el Diseño	0/1	0
MA Medio Ambiente	0/0	0
PTR Procesos y Técnicas de Realización	0/1	0

Requisitos de Recuperación
26-I

CUMPLIMIENTO POR DEPARTAMENTO

DEPARTAMENTO	PROFESORES	REQUISITO
EDT Evaluación del Diseño en el Tiempo	0/0	0
ICD Investigación y Conocimiento para el Diseño	0/1	0
MA Medio Ambiente	0/0	0
PTR Procesos y Técnicas de Realización	0/1	0

Autoevaluación
26-I

Autoevaluación Docente 26-I

PUBLICADO

Figura 3. Dashboard del administrador tras el inicio de sesión: 2 profesores activos, 1 formulario publicado, tablas de cumplimiento por departamento inicializadas.

Autoevaluación
Formularios de evaluación docente.

+ Nuevo Formulario

FORMULARIO	ESTADO	PREGUNTAS	RESPUESTAS	
Autoevaluación Docente 26-I 26-I	PUBLICADO	6	0	Ver Editar Duplicar Eliminar

Figura 4. Módulo de Autoevaluación (rol admin) — formulario *Autoevaluación Docente 26-I* en estado PUBLICADO, 6 preguntas.

UEA

Importar CSV + Nueva UEA

Buscar: Por clave o nombre... Programa: — Todos —

CLAVE	UEA	PROGRAMA	ÁREA	TIPO	ESTADO	
1200015	Tipografía Aplicada	Diseño de la Comunicación Gráfica	Licenciatura	OBL	ACTIVO	Ver Editar Eliminar
1100001	Introducción al Diseño	Arquitectura	Licenciatura	OBL	ACTIVO	Ver Editar Eliminar
1403004	Taller de Diseño Industrial I	Diseño Industrial	Licenciatura	OBL	ACTIVO	Ver Editar Eliminar
1500020	Sustentabilidad y Proyecto	Diseño de Proyectos Sustentables	Licenciatura	OBL	ACTIVO	Ver Editar Eliminar
9000001	Seminario de Diseño Bioclimático I	Posgrado en Diseño Bioclimático	Licenciatura	OBL	ACTIVO	Ver Editar Eliminar

Figura 5. Catálogo de UEA con las 5 unidades sembradas por `seed_demo.py` y filtro por programa académico.

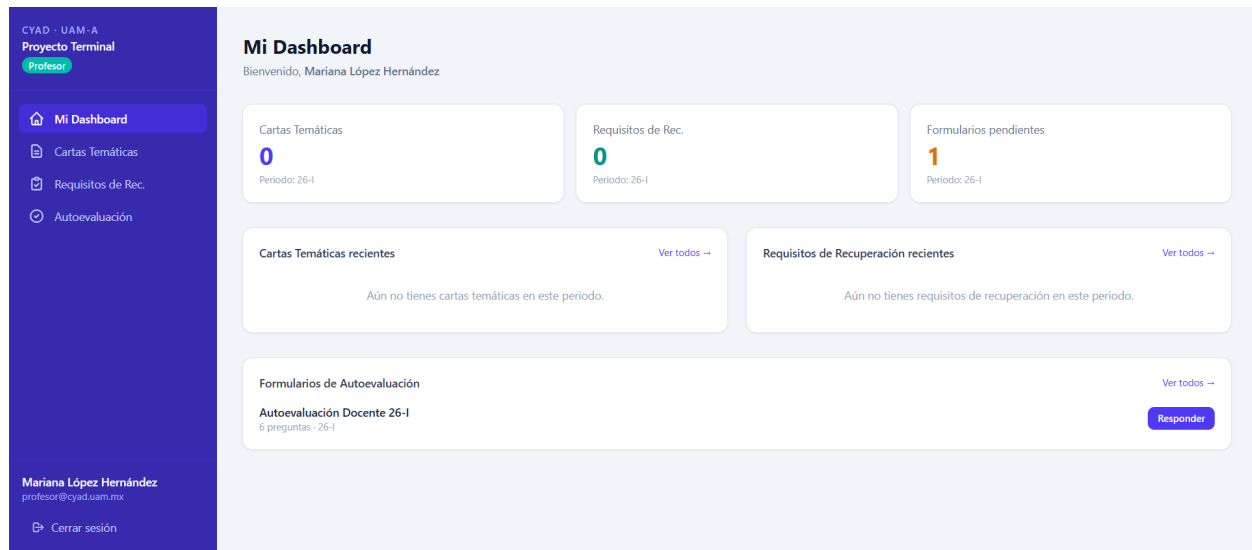


Figura 6. Dashboard del profesor (Mariana López Hernández) — 1 formulario pendiente correctamente listado.

Figura 7. Formulario de autoevaluación listo para responder — dos secciones, 6 preguntas de tipos distintos (opción única, casillas, sí/no, escala lineal, opción con selector, texto largo).

7. Smoke test

```
$ python scripts/smoke_test.py
=== Salud del backend ===
[OK ] GET /health → 200

=== Login ===
```

```

[OK  ] POST /auth/login admin → 200
[OK  ] POST /auth/login profesor → 200

=== /auth/me ===
[OK  ] me admin → rol=ADMIN
[OK  ] me profesor → rol=PROFESOR

=== Catálogos ===
[OK  ] GET /uea/ → 200 (count=5)
[OK  ] GET /periodos/ → periodo activo 26-I

=== Autoevaluación -- flujo profesor ===
[OK  ] GET /formularios-disponibles/ → 200 (count=1)
[OK  ] Detalle formulario v1 (6 preguntas)
[OK  ] Respuesta (borrador o existente) id=1

=== Documentos ===
[OK  ] POST /cartas-tematicas/ → 201 -- id=1

=== Reportes (admin) ===
[OK  ] GET /reportes/dashboard/ → 200
[OK  ] GET /reportes/cumplimiento-licenciatura/ → 200

=====
SMOKE TEST: TODOS LOS CHECKS OK
real  0m30.774s

```

8. Suite pytest

Se ejecutó la suite completa como paso opcional (`pytest -q desde backend/`):

```

$ .venv/Scripts/python.exe -m pytest -q
...
FAILED tests/test_reportes.py::TestDashboard::
      test_dashboard_estructura_autoevaluacion
FAILED tests/test_reportes.py::TestResumenAutoevaluacion::
      test_tasa_respuesta_calculada
FAILED tests/test_reportes.py::TestAutoevaluacionProfesores::
      test_profesor_sin_respuesta_0_porcentaje
FAILED tests/test_reportes.py::TestAutoevaluacionProfesores::
      test_profesor_con_respuesta_enviada
4 failed, 192 passed, 3 warnings in 406.39s (0:06:46)

```

Los 4 tests que fallan pertenecen todos a `tests/test_reportes.py` y comparten la misma causa raíz: el *fixture* de `test` intenta publicar un formulario que —al igual que

`seed_demo.py`— construye sus secciones sin peso, por lo que la validación de la clase `Formulario` rechaza la publicación con `ValueError`: Debe definir al menos una sección antes de publicar (o su variante “suma de pesos = 0 %”). El resto de la suite (192/196 = 98 %) pasa limpia sobre la base de datos recién migrada.

Esto se registra como hallazgo **H10** y confirma que el bug de H5 vive tanto en el seed como en los *fixtures* de tests de reportes; corregir el modelado de secciones con peso inicial hará que ambos escenarios verdes.

9. Hallazgos y discrepancias

ID	Hallazgo	Ubicación original	Corrección
H1	PostgreSQL: manual pedía v.16 mínima; máquina y README dicen 14+. La prueba corrió sobre 17.	<code>main.tex</code> , tabla de pre-requisitos.	Alineado a “14+ (probado con 17)”.
H2	<code>psql</code> no queda en <code>PATH</code> tras la instalación oficial en Windows. El manual asumía lo contrario.	<code>main.tex</code> , sección Pre-requisitos y Solución de problemas.	Añadida nota de <code>PATH</code> y alternativa <code>create_db.py</code> .
H3	El <code>.env.example</code> de la raíz define <code>ACCESS_TOKEN_LIFETIME/REFRESH_TOKEN_LIFETIME</code> , pero <code>settings.py</code> lee <code>JWT_ACCESS_MINUTES/JWT_REFRESH_DAYS</code> . Es una plantilla obsoleta.	<code>/.env.example</code> vs <code>backend/config/settings.py</code>	Manual ahora indica que se use <code>backend/.env.example</code> . Se recomienda actualizar el archivo raíz en un cambio posterior.
H4	README instruye <code>cp .env.example .env</code> en la raíz, pero Django lee el <code>.env</code> desde <code>backend/</code> .	<code>README.md</code> sección 3.1 y <code>main.tex</code> sección Backend/Archivo <code>.env</code> .	Manual aclara ubicación correcta (<code>backend/</code>) y nombra <code>backend/.env.example</code> como plantilla oficial.
H5	Bug real: <code>seed_demo.py</code> crea Sección sin campo peso, por lo que <code>formulario.publicar()</code> lanza <code>ValueError</code> en la validación <code>_validar_estructura_para_publicar</code> .	<code>backend/scripts/seed_demo.py</code> línea 297–303.	<i>Workaround</i> local: agregar <code>peso=Decimal("50")</code> a ambas secciones. Manual ahora lista el error y la corrección. Recomendación: corregir el script en un commit dedicado.

ID	Hallazgo	Ubicación original	Corrección
H6	Manual usa <code>VITE_API_BASE_URL</code> pero el código lee <code>VITE_API_URL</code> (<code>frontend/src/api/client.js</code> , <code>publico.js</code>).	<code>main.tex</code> sección Frontend.	Corregido a <code>VITE_API_URL</code> con nota explicativa.
H7	<code>npm install</code> reporta 8 vulnerabilidades transitorias (1 moderada, 7 altas). No impide el arranque.	<code>frontend/package-lock</code>	Documentado. Correr <code>npm audit fix</code> o actualizar dependencias en un mantenimiento futuro.
H8	Vite advierte que el <i>bundle</i> JS supera 500 kB tras minificar (893 kB, 263 kB gzip). No es error.	<code>frontend build</code> .	Considerar <code>code-splitting</code> con <code>import()</code> dinámico o <code>build.rollupOptions.output</code> . Registrado para trabajo futuro.
H9	Discrepancia menor en versiones mínimas entre manual y README (Python 3.13 vs 3.11+; Node 22 LTS vs 20+).	<code>main.tex</code> <code>README.md</code> .	vs Manual alineado con README y anotado como “probado con”.
H10	4 de 196 tests <i>pytest</i> fallan por la misma causa raíz que H5: los <i>fixtures</i> de <code>tests/test_reportes.py</code> publican formularios sin definir peso en las secciones. Suite verde restante: 192/196 (98 %).	<code>backend/tests/test_reportes.py</code> y modelo Sección.	Corregir junto con H5: hacer que los <i>fixtures</i> de test asignen <code>peso=Decimal("100")</code> (una sola sección) o distribuyan pesos que sumen 100.

10. Recomendaciones

1. **Corregir `seed_demo.py`** añadiendo `peso=Decimal("50")` a ambas secciones — el *workaround* de este documento debe promoverse a un commit permanente para que futuros clones no requieran intervención manual.
2. **Retirar o alinear `/.env.example`** raíz con el esquema de `settings.py` (variables `JWT_ACCESS_MINUTES`/`JWT_REFRESH_DAYS`). Actualmente induce a error al lector que sigue el README literalmente.
3. **Actualizar el README** en la sección “Configuración rápida” para dejar claro que

el `.env` vive en `backend/`, no en la raíz.

4. **Agregar un script de bootstrap** (por ejemplo `scripts/bootstrap.ps1` o un objetivo `make setup`) que orqueste `create_db.py`, `migrate` y `seed_demo.py` en un único comando, reduciendo la superficie de error humano.
5. **Considerar *code-splitting* en Vite** para reducir el tamaño inicial del *bundle*, mejorando el tiempo de primera carga de la interfaz.
6. **Auditar dependencias JS** (`npm audit fix`) antes de la defensa; hoy hay 7 vulnerabilidades altas transitorias.

11. Veredicto

Tras aplicar el *workaround* de `seed_demo.py` (hallazgo H5), el proyecto se monta completamente desde cero siguiendo el manual actualizado en menos de 3 minutos de tiempo humano (aprox. 2 min de trabajo de máquina), y las 12 verificaciones del *smoke test* pasan sobre una base de datos recién creada. Los flujos de administrador y profesor son navegables y el formulario de autoevaluación queda listo para ser respondido. El sistema es reproducible.